Arithmetic Reasoning Under Distribution Shift

An Evaluation of Generalization in a Small LLM

By- Anushka De

Stat 359: Final Project Report

Introduction

Large language models (LLMs) have demonstrated impressive capabilities across a wide range of tasks, including natural language understanding, text generation, and reasoning. Among these capabilities, arithmetic reasoning has emerged as an important benchmark for evaluating whether language models can perform structured, step-by-step computations rather than simply recognizing statistical patterns in text. Arithmetic tasks provide a controlled setting in which the correctness of model outputs can be objectively verified and reasoning processes can be analyzed in detail.

Despite strong performance on tasks similar to those seen during training, many models struggle to generalize beyond their training distribution. This issue becomes particularly evident in reasoning tasks, where a model may perform well on familiar patterns but fail when confronted with slightly different inputs. In arithmetic reasoning, such distribution shifts can arise from changes in the magnitude of operands or the structural complexity of expressions. Understanding how models behave under these conditions provides valuable insights into whether they have learned generalizable computational procedures or merely memorized patterns within a limited range of examples.

In this project, we study the behavior of a small transformer-based arithmetic language model trained on synthetically generated arithmetic expressions involving addition and subtraction. The training data consists of expressions with operands ranging from 1 to 20 and a maximum expression depth of four operations. After training the model on this distribution, we evaluate its performance under several types of out-of-distribution (OOD) settings. Specifically, we examine how model accuracy changes when the operand range increases (e.g., 1–99 and 1–999) and when the expression depth becomes larger than those seen during training.

To better understand the nature of model failures, we perform multiple analyses including accuracy comparisons, error type breakdowns, and error magnitude evaluation. These analyses allow us to examine not only how often the model fails but also how severely its predictions deviate from the correct results. Through this systematic evaluation, the project aims to determine whether the arithmetic LLM exhibits genuine algorithmic reasoning or primarily relies on patterns learned within the training distribution.

Overall, this study provides a controlled investigation of generalization and robustness in arithmetic reasoning, highlighting how distribution shifts in numerical magnitude and expression complexity affect the behavior of language models.

Methodology and Experimental Setup

Model Overview

The experiments in this project use a small transformer-based language model designed to evaluate arithmetic expressions. The model is trained to generate step-by-step reasoning traces followed by a final numerical result. Each training example consists of an arithmetic expression and a structured solution format containing intermediate computation steps enclosed within <think> tokens, followed by the final answer.

The model architecture is a lightweight transformer that predicts tokens autoregressively. During inference, the model receives an arithmetic expression as input and generates a sequence of reasoning steps leading to a final result.

Training Data

The training dataset consists of synthetically generated arithmetic expressions involving addition and subtraction. Expressions are generated programmatically to ensure correctness and to control their structural complexity.

The training distribution has the following characteristics:

- **Number of training samples:** 100,000
- **Operations:** addition (+) and subtraction (−)
- **Operand range:** integers from 1 to 20
- **Maximum expression depth:** 4

Examples of training expressions include:

1 + (2 + (3 + (4 + 5)))
((2 + 5) - (3 - 1)) + 4
6 + (3 + (4 + (2 − 1)))

For each expression, the training data includes a step-by-step reasoning trace describing the intermediate computations required to evaluate the expression.

Evaluation Setup

After training, the model is evaluated on newly generated test expressions. The evaluation dataset consists of **1,000 expressions** generated using the same expression generator but under different parameter settings.

The evaluation focuses on how the model behaves under **distribution shift**, where the test inputs differ from the training distribution in controlled ways.

Two types of distribution shifts are considered:

Operand Range Shift

To test numerical generalization, the operand range used to generate expressions is increased beyond the training range.

The following ranges are evaluated:

- **1–20** (in-distribution)
- **1–99** (moderate out-of-distribution)
- **1–999** (strong out-of-distribution)

This experiment measures how model performance changes when it encounters numbers larger than those seen during training.

Expression Depth Shift

To evaluate reasoning complexity, the maximum depth of arithmetic expressions is increased.

The following depths are tested:

- **Depth 4** (training distribution)
- **Depth 6** (moderately deeper expressions)
- **Depth 8** (significantly deeper reasoning chains)

This experiment evaluates whether the model can generalize to expressions requiring longer reasoning sequences.

Combined Distribution Shift

To examine the most challenging conditions, both operand magnitude and expression depth are increased simultaneously. This allows us to evaluate how the model behaves when it encounters **multiple forms of distribution shift at the same time**.

Evaluation Metrics

Model performance is measured using several quantitative metrics.

Exact Match Accuracy

Exact match accuracy measures the percentage of expressions for which the model's predicted final result exactly matches the correct answer.

Error Type Analysis

Model outputs are categorized into different error types, including:

- Correct predictions
- Incorrect arithmetic results
- Non-parseable outputs

This analysis helps identify the primary sources of model failure.

Error Magnitude

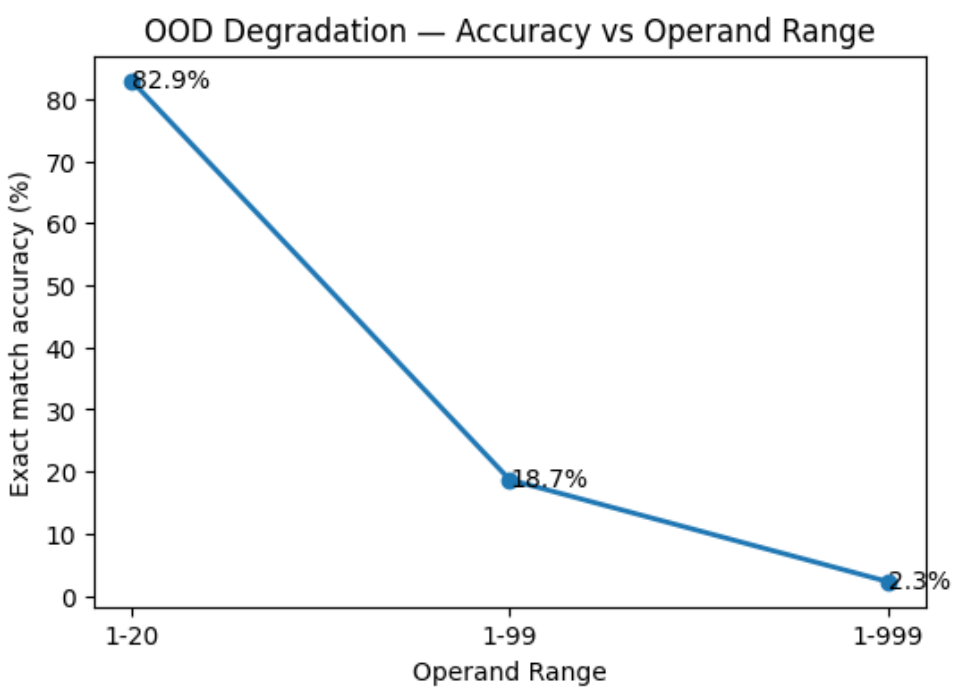
To quantify how incorrect the model's predictions are, we compute the **absolute error magnitude** between the predicted result and the ground truth:

$$|\,Predicted\ Result - Ground\ Truth|$$

This analysis provides insight into how prediction errors grow as expressions become more challenging.

Results and Discussion

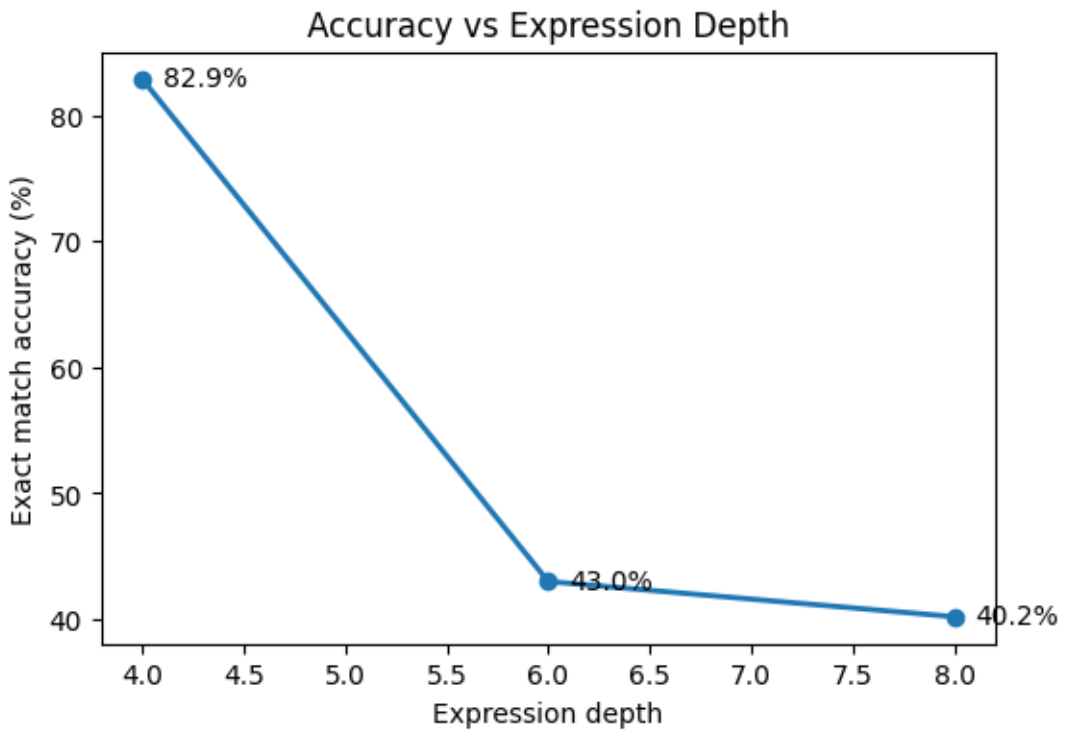
Comparison Across Operand Range



The plot illustrates how the model's exact match accuracy changes as the operand range increases while keeping the expression structure constant, with the maximum expression depth fixed at 4. The results show that the model performs very well within the training range. When the operands lie between 1 and 20, the model achieves an accuracy of 82.9%, indicating strong performance on in-distribution data that closely resembles the data used during training. However, the performance drops sharply when the operand range is expanded. Increasing the range to 1–99 causes the accuracy to decline drastically to 18.7%, even though the structure of the arithmetic expressions remains unchanged. This suggests that the model struggles to maintain performance when the numerical values become larger than those encountered during training. The degradation becomes even more severe when the operand range is extended further to 1–999, where the accuracy falls to just 2.3%, which is close to random performance.

These results highlight a clear case of out-of-distribution (OOD) degradation. Although the compositional structure of the expressions remains the same, the increase in numerical magnitude significantly harms the model's performance. This behavior suggests that the model may be relying on memorized patterns within a limited numeric range rather than learning a fully general arithmetic reasoning mechanism. Overall, the findings indicate that operand magnitude plays a crucial role in the model's ability to generalize, and the model's reasoning capability does not scale effectively to larger numbers despite being able to handle similar expression structures during training.

Comparison Across Expression Depth

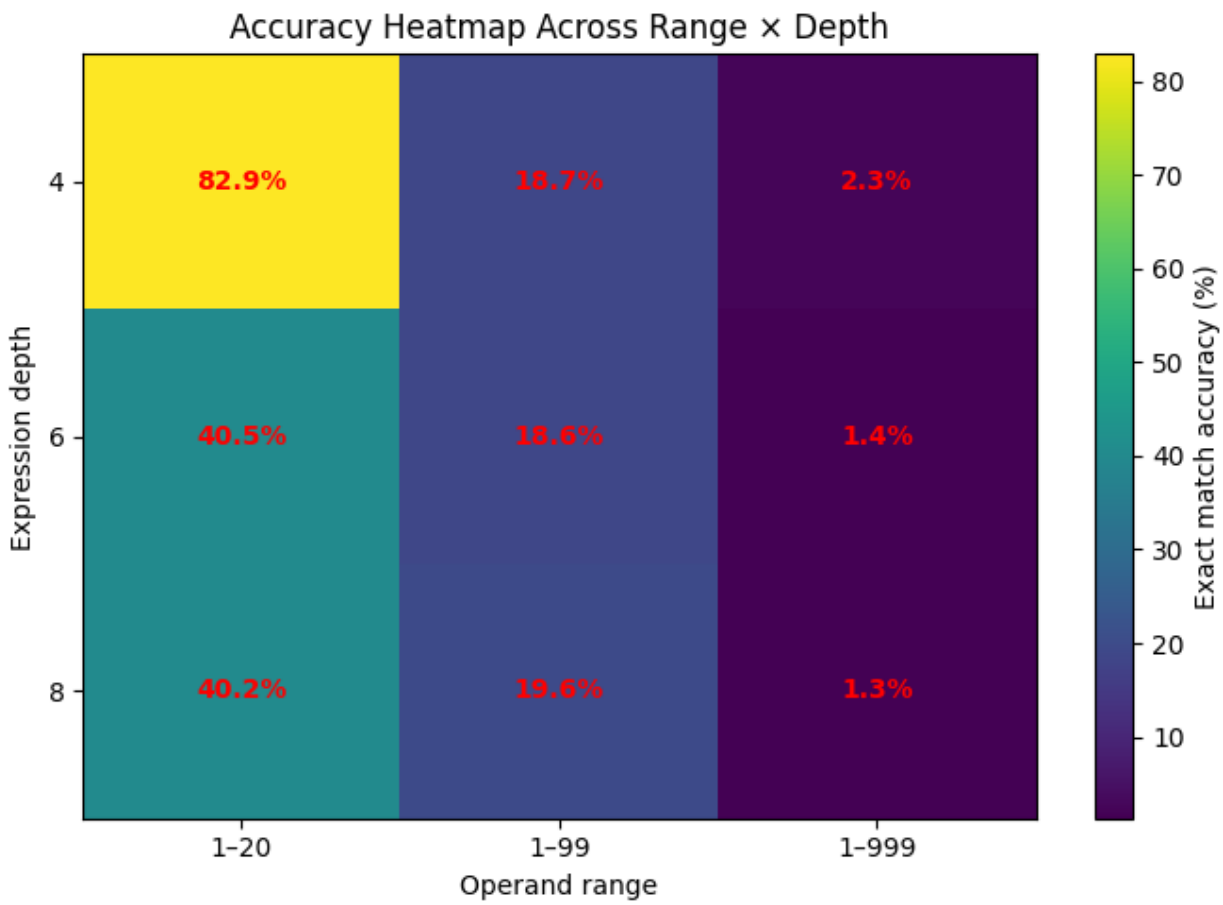


The plot illustrates how the model's exact match accuracy changes as the expression depth increases, while the range of operands remains fixed between 1 and 20. The model was trained only on expressions with a maximum depth of 4, making larger depths out-of-distribution in terms of structural complexity. The results show that the model performs very well at the training depth. When the expression depth is 4, the model achieves an accuracy of 82.9%, indicating strong performance on in-distribution data where the

complexity of the expressions matches the training setup. However, the performance drops considerably as the depth increases. When the depth increases to 6, the accuracy decreases sharply to 43.0%, suggesting that the model struggles to generalize to longer sequences of operations even though the underlying arithmetic rules remain unchanged. When the depth is further increased to 8, the accuracy declines slightly to 40.2%. Although the model is still able to solve some expressions correctly, the overall performance remains substantially lower than at the training depth.

These results suggest that increasing the expression depth increases the number of sequential reasoning steps required to correctly evaluate the expression. Because the model was trained only on expressions with depth up to 4, it likely learned patterns specific to shorter reasoning chains rather than a fully general evaluation procedure. Consequently, when the model encounters deeper expression trees, its performance deteriorates, reflecting a limitation in its ability to generalize compositionally. Overall, the plot demonstrates that expression depth plays a significant role in determining model performance. While the model performs well on expressions that match the training complexity, its accuracy decreases substantially when expressions require additional reasoning steps, highlighting limited generalization to more complex compositional structures.

Combined OOD Analysis



The heatmap summarizes how the model's exact match accuracy varies jointly with operand range and expression depth. The results show that the best performance occurs in the training setting, where the expression depth is 4 and the operand range is 1–20. In this configuration, the model achieves the highest

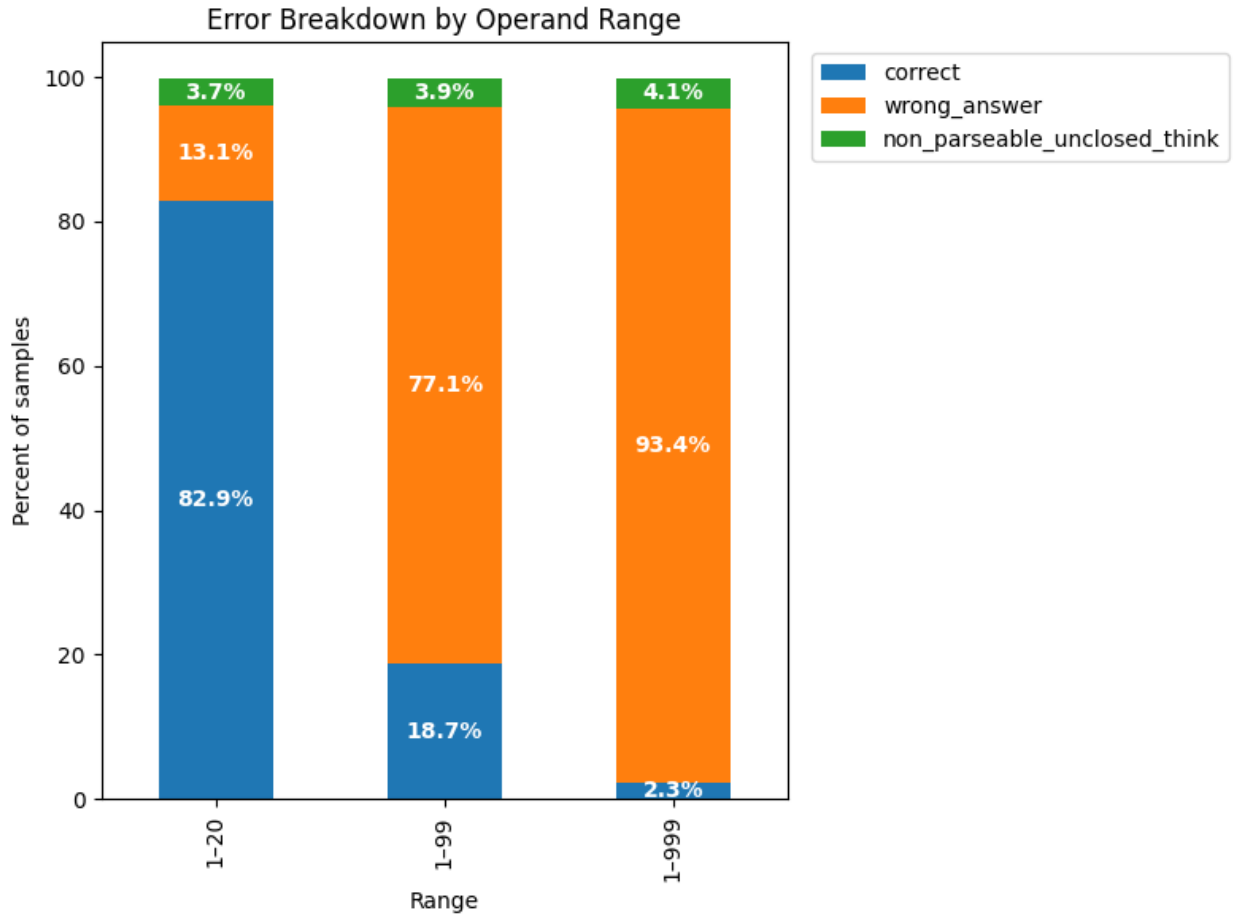
accuracy of 82.9%, indicating that it performs well when both the numeric magnitude of operands and the structural complexity of expressions match the training distribution.

However, the heatmap also reveals a strong degradation in performance as the operand range increases. When the operand range expands from 1–20 to 1–99, the accuracy drops sharply to approximately 18–19%. When the range is further increased to 1–999, accuracy declines even more dramatically to around 1–2%. This pattern is consistent across all expression depths, suggesting that the model struggles significantly to generalize to larger numeric magnitudes.

Increasing the expression depth also negatively affects performance, though to a lesser extent. Within the small operand range (1–20), increasing the depth from 4 to 6 and then to 8 reduces accuracy from 82.9% to roughly 40%. This indicates that deeper expressions, which require longer chains of reasoning, are more difficult for the model to evaluate. However, once the operand range becomes large, increasing depth has little additional effect because the accuracy is already extremely low.

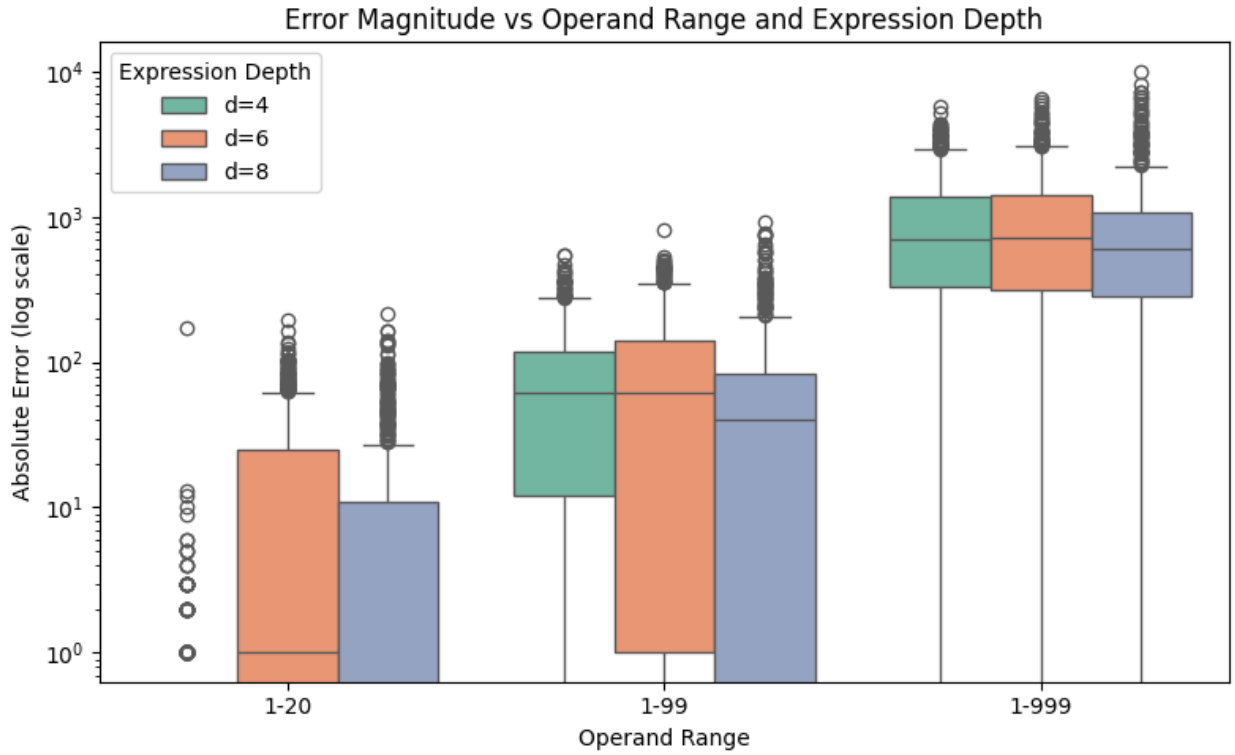
Overall, the heatmap indicates that operand magnitude has a stronger impact on model performance than expression depth. For example, increasing the depth from 4 to 8 within the 1–20 range reduces accuracy by about 40 percentage points, whereas expanding the operand range from 1–20 to 1–999 at depth 4 reduces accuracy by roughly 80 percentage points. These results highlight two sources of out-of-distribution degradation: structural complexity, which moderately reduces performance, and numeric magnitude, which leads to a much more severe decline. This suggests that the model relies heavily on patterns within the numeric range seen during training rather than learning a fully general arithmetic procedure that scales effectively to larger numbers or deeper expressions.

Error Type Breakdown



This plot reinforces the earlier findings on out-of-distribution degradation: increasing the operand range dramatically increases the proportion of wrong answers, while formatting errors remain minimal. This indicates that the main limitation lies in the model's numerical reasoning and generalization to larger numbers, rather than in generating properly formatted outputs.

Error Magnitude



The plot illustrates the distribution of absolute errors (shown on a logarithmic scale) for incorrect predictions across different operand ranges and expression depths. The results indicate that the magnitude of errors increases substantially as the operand range grows. When the operand range is 1–20, most errors remain relatively small, with many falling between 1-10^2. However, when the range expands to 1–99, the distribution shifts upward, with many errors appearing between 10^2 and 10^3. For the largest operand range, 1–999, the errors become significantly larger, often reaching values between 10^3 and 10^4. This pattern suggests that when the model fails on problems involving larger numbers, the numerical deviation from the correct result increases considerably.

The plot also shows that expression depth has a comparatively smaller effect on the magnitude of errors. Across depths 4, 6, and 8, the error distributions within the same operand range remain relatively similar. Although deeper expressions introduce slightly more variability in the errors, the primary factor influencing error size is the operand magnitude rather than the structural complexity of the expression. Additionally, the boxplots reveal long upper tails, particularly for the 1–999 operand range, indicating the presence of extreme outliers. These large deviations suggest that the model sometimes breaks down entirely when performing multi-step arithmetic with large operands, producing predictions that are far from the correct values.

Overall, the results reinforce earlier findings regarding out-of-distribution degradation. As the operand range increases, both the frequency and the magnitude of errors grow significantly, while expression depth has a comparatively smaller impact on error size. This indicates that the model struggles much more with scaling to larger numerical magnitudes than with handling deeper expression structures.

Conclusion

In this project, we investigated the robustness of a transformer-based arithmetic language model under **out-of-distribution (OOD) conditions**. The model was trained on synthetically generated arithmetic expressions containing addition and subtraction with operands ranging from **1 to 20** and expression depth up to **four operations**. We then evaluated the model under controlled distribution shifts involving **larger operand ranges** and **greater expression depths**.

The experimental results reveal a clear limitation in the model's ability to generalize beyond its training distribution. While the model achieves moderate performance within the training range, accuracy declines sharply when the operand magnitude increases or when expressions require longer reasoning chains. In particular, increasing the operand range from **1–20 to 1–999** results in a dramatic drop in accuracy, suggesting that the model struggles to handle numbers that differ significantly from those seen during training. Similarly, deeper expressions lead to more frequent reasoning failures due to error accumulation across intermediate computation steps.

Further analysis of model outputs shows that most failures arise from **incorrect arithmetic reasoning rather than formatting errors**. The model typically generates outputs that follow the expected reasoning structure but performs incorrect intermediate computations. Additionally, the **error magnitude analysis** indicates that prediction errors become substantially larger as both operand magnitude and expression depth increase, demonstrating that the model's predictions diverge significantly from the correct results under strong distribution shifts.

The experimental results reveal a clear limitation in the model's ability to generalize beyond its training distribution. While the model achieves moderate performance within the training range, accuracy declines sharply when the operand magnitude increases or when expressions require longer reasoning chains. In particular, increasing the operand range from **1–20 to 1–999** results in a dramatic drop in accuracy, suggesting that the model struggles to handle numbers that differ significantly from those seen during training. Similarly, deeper expressions lead to more frequent reasoning failures due to error accumulation across intermediate computation steps.

Further analysis of model outputs shows that most failures arise from **incorrect arithmetic reasoning rather than formatting errors**. The model typically generates outputs that follow the expected reasoning structure but performs incorrect intermediate computations. Additionally, the **error magnitude analysis** indicates that prediction errors become substantially larger as both operand magnitude and expression depth increase, demonstrating that the model's predictions diverge significantly from the correct results under strong distribution shifts.

Overall, this project highlights the importance of evaluating language models under **controlled distribution shifts**. Strong performance within the training distribution does not necessarily imply robust reasoning ability, and careful evaluation under varied conditions is essential for understanding the true capabilities and limitations of these models.